МДК.02.01 Технология разработки программного обеспечения

CASE-средства

Общие сведения о CASE-средствах

Большие размеры и высокая сложность разрабатываемого программного обеспечения при ограничениях на финансовые и временные ресурсы проекта могут привести к низкому качеству конечных программных продуктов и системы в целом.

В этой связи в последнее время все большее внимание уделяется ***современным технологиям и инструментальным средствам***, ***обеспечивающим автоматизацию процессов жизненного цикла программного обеспечения (CASE-средствам).***

Использование таких инструментальных средств позволяет существенно ***сократить длительность и стоимость разработки систем при одновременном повышении качества процесса разработки*** и, как следствие, качества разработанных программных средств.

CASE (Computer Aided Software/System Engineering) — ***набор инструментов и методов программной инженерии для проектирования программного обеспечения, который помогает обеспечить высокое качество программ, отсутствие ошибок и простоту в обслуживании программных продуктов.***

Также под CASE понимают ***совокупность методов и средств проектирования информационных систем с использованием CASE-инструментов.***

Средства автоматизации разработки программ (CASE-средства) — инструменты автоматизации процессов проектирования и разработки программного обеспечения ***для системного аналитика, разработчика ПО.***

Первоначально термин CASE трактовался как Computer Aided Software Engineering (*компьютерная поддержка проектирования программного обеспечения).*

Но с приходом стандарта ISO/IEC 14102 CASE-средства стали определять ***как программные средства для поддержки процессов жизненного цикла ПО***.

В настоящее время данному термину придается еще более широкий смысл, и он расшифровывается как **Computer Aided System Engineering** ***(компьютерная поддержка проектирования систем)***.

Современные CASE-средства ориентируются на моделирование предметной области, разработку спецификаций, проектирование сложных систем широкого назначения.

CASE-технология — совокупность методологий разработки и сопровождения программных средств и сложных систем, поддерживаемая комплексом взаимосвязанных средств автоматизации.

Основные цели использования CASE-технологий при разработке программного обеспечения — отделить анализ и проектирование от программирования и последующих работ процесса разработки, предоставив разработчику соответствующие методологии визуального анализа и проектирования.

CASE-средства — любые программные средства, автоматизирующие ту или иную совокупность процессов жизненного цикла ПО, а именно: анализ и формулировку требований, проектирование баз данных и приложений, генерацию кода, тестирование, обеспечение качества, управление конфигурацией и проектом.

CASE-систему можно определить как набор CASE-средств, имеющих определенное функциональное предназначение и выполненных в рамках единого программного продукта.

Современные CASE-средства охватывают обширную область поддержки многочисленных технологий проектирования информационных систем от простых средств анализа и документирования до полномасштабных средств автоматизации.

Их использование позволяет не только ускорить работы и повысить качество их выполнения, но и дает инструменты ***для организации коллективного труда группы проектировщиков.***

Принципы построения и приемы работы с CASE-средствами

Фактически CASE-средство — совокупность графически ориентированных инструментальных средств, поддерживающих процессы или отдельные этапы процессов жизненного цикла программного обеспечения.

К CASE-средствам может быть отнесено любое программное средство, обеспечивающее автоматическую помощь при разработке ПС, их сопровождении или управлении проектом, базирующееся на следующих **основополагающих принципах**:

- наличие мощных графических средств для описания и документирования системы, обеспечивающих удобный интерфейс с разработчиком и использующих его творческие возможности;
- интеграция отдельных компонентов CASE-средств, обеспечивающая управление процессом разработки программного обеспечения;
- использование специальным образом организованного хранилища проектных метаданных — репозитория.

В основе концептуального построения CASE-средств лежат следующие положения:

- учет человеческого фактора, позволяющий привести процессы жизненного цикла программного обеспечения к легкой, удобной и экономичной форме;
- использование базовых программных средств, применяющихся в других приложениях (СУБД, компиляторы с различных языков программирования, отладчики и др.);
- автоматизированная или автоматическая кодогенерация;
- ограничение сложности, позволяющее поддерживать сложность компонентов разрабатываемого программного средства или системы на уровне, доступном для понимания, использования и модификации;
- доступность для различных категорий пользователей, в том числе заказчиков, специалистов в предметной области, системных аналитиков, проектировщиков, программистов, тестировщиков, инженеров по качеству, менеджеров проектов;
- рентабельность, обеспечивающая быструю окупаемость денежных средств, вложенных в приобретение CASE-средства, за счет сокращения сроков и стоимости проектов;
- сопровождаемость. CASE-средства обладают способностью адаптации к изменяющимся требованиям и целям проекта.

CASE-технологии призваны максимально автоматизировать процессы разработки и сопровождения программного продукта.
Они обеспечивают качество принимаемых технических решений и подготовку проектной документации.
Графические средства позволяют разработчикам в наглядной форме моделировать предметную область, изучать существующую информационную систему, изменять ее в соответствии с поставленными целями и имеющимися ограничениями.

Методы визуального представления информации играют при этом очень важную роль и выбираются в соответствии с информационными потребностями пользователей.

Основные функциональные возможности CASE-средств

Основные компоненты CASE-средств и их функции

CASE-средства содержат инструменты различного функционального назначения, поддерживающие различные этапы основных, вспомогательных и организационных процессов жизненного цикла программного обеспечения. В состав CASE-средств входят четыре **основных компонента**.

- **Средства централизованного хранения всей информации о проекте — репозиторий.** Предназначен для хранения информации о разрабатываемом программном обеспечении или системе в течение всего жизненного цикла разработки.
- **Средства ввода.** Служат для ввода данных в репозиторий, организации взаимодействия участников проекта с CASE-средством. Должны поддерживать различные методологии анализа, проектирования, тестирования, контроля. Предназначены для использования в течение жизненного цикла программного обеспечения или системы различными категориями участников проекта (системными аналитиками, проектировщиками, программистами, тестировщиками, менеджерами, специалистами по качеству и т. д.).
- **Средства анализа и разработки.** Предназначены для анализа различных видов графических и текстовых описаний и их преобразований в процессе разработки.
- **Средства вывода.** Служат для кодогенерации, создания различного вида документов, управления проектом

Все компоненты CASE-средств в совокупности обладают следующими **функциональными возможностями**:

- поддержкой графических моделей;
- контролем ошибок;
- поддержкой репозитория;
- поддержкой основных, вспомогательных и организационных процессов ЖЦ ПС.

Поддержка графических моделей в CASE-средствах

Наиболее трудоемкими стадиями разработки программного обеспечения являются ***стадии формирования требований и проектирования,*** в процессе которых CASE-средства обеспечивают качество принимаемых технических решений и подготовку проектной документации.

При этом большую роль играют ***методы визуального представления информации***.

Графические средства анализа и проектирования обеспечивают создание и редактирование комплекса взаимосвязанных диаграмм, образующих модели системы ПО.

Возможны создание, перемещение и выравнивание групп объектов, изменение их размеров, масштабирование, сохранение связей между объектами при их перемещении и изменении размеров, автоматический контроль ошибок и др.

Графические средства моделирования предметной области позволяют разработчикам в наглядном виде изучать существующую информационную систему, перестраивать ее в соответствии с поставленными целями и имеющимися ограничениями.

На разных уровнях проектирования могут использоваться различные виды и нотации графического представления программного обеспечения.

Обычно применяются диаграммы различных типов, в том числе иерархии требований.

Разработка диаграмм осуществляется с помощью специальных графических редакторов, основными функциями которых являются создание и редактирование иерархически связанных диаграмм, их объектов и связей между объектами, а также автоматический контроль ошибок.

Контроль ошибок в CASE-средствах

На стадиях формирования требований и проектирования очень важен контроль ошибок. Это связано с тем, что на более поздних стадиях их выявление и устранение обходятся значительно дороже.

CASE-технология обеспечивает автоматическую верификацию и контроль проекта на полноту и состоятельность на ранних этапах разработки, что влияет на успех разработки в целом. В CASE-средствах обычно реализуются следующие **виды контроля**:

- *контроль синтаксиса* диаграмм и типов их элементов;
- *контроль полноты и состоятельности диаграмм* (все элементы диаграмм должны быть идентифицированы и отражены в репозитории);
- *сквозной контроль* диаграмм одного или различных типов на предмет их состоятельности по уровням — *вертикальное и горизонтальное балансирование диаграмм*. Вертикальное балансирование диаграмм одного типа подразумевает соответствие потоков данных между детализируемой и детализирующей диаграммами. Горизонтальное балансирование определяет несоответствия между структурами данных и спецификациями процессов;
- *контроль декомпозиции функций*. При данном типе контроля выполняется оценка декомпозиции на основе различных метрик. Например, может быть оценена эффективность и корректность декомпозиции с точки зрения связности и сцепления модулей.

Поддержка репозитория в CASE-средствах

Репозиторий обеспечивает хранение версий проекта и его отдельных компонентов, синхронизацию поступления информации от различных разработчиков при групповой разработке, контроль метаданных на полноту и непротиворечивость.

Содержимое репозитория включает не только информационные объекты различных типов, но и правила использования или обработки этих компонентов.

Репозиторий может хранить диаграммы, определения экранов и меню, проекты отчетов, описания данных, исходные коды и т. д. Каждый информационный объект в репозитории описывается перечислением его свойств (идентификатор, синоним, тип, текстовое описание, компоненты, область значений и т. д.).

Кроме того, в нем хранятся правила формирования и редактирования объекта, а также контрольная информация о времени создания объекта, времени его последнего обновления, номере версии, возможности обновления и т. д.

Репозиторий является базой для стандартизации документации по проекту и контроля проектных спецификаций. Все отчеты строятся автоматически по содержимому репозитория.

Несомненное достоинство CASE-технологии заключается в том, что документация всегда отражает текущее состояние дел, поскольку любые изменения в проекте автоматически отражаются в репозитории.

Посредством репозитория может осуществляться контроль безопасности (ограничения доступа, привилегии доступа), контроль версий, контроль изменений и др.

Репозиторий является базой для автоматической генерации документации по проекту. **Основными типами отчетов** являются:

- **отчеты по содержимому** — включают информацию по потокам данных и их компонентов; списки функциональных блоков диаграмм и их входных и выходных потоков; списки всех информационных объектов и их атрибутов; историю изменений объектов; описания модулей и интерфейсов между ними; планы тестирования модулей и т. д.;
- **отчеты по перекрестным ссылкам** — содержат информацию по связям всех вызывающих и вызываемых модулей; списки объектов репозитория, к которым имеет доступ конкретный исполнитель проекта; информацию по связям между диаграммами и конкретными данными; маршруты движения данных от входа к выходу;
- **отчеты по результатам анализа** — включают данные по взаимной корректности диаграмм, списки неопределенных информационных объектов, списки неполных диаграмм, данные по результатам анализа структуры проекта и др.;
- **отчеты по декомпозиции объектов** — включают совокупности объектов, а именно: таблицы иерархии всех объектов модели.

Поддержка процессов жизненного цикла программных средств и систем

Основой поддержки процесса разработки являются представленные далее **свойства современных CASE-средств**.

1. **Покрытие всего жизненного цикла систем или программных средств.** Современные CASE-средства поддерживают практически полный жизненный цикл программного обеспечения. Однако первоочередное внимание уделяется начальным работам процесса разработки — анализу требований к системе, проектированию системной архитектуры, анализу требований к программным средствам и проектированию программной архитектуры.

Грамотная разработка требований к программному обеспечению является основой всего проекта, их полнота и корректность определяют уровень соответствия результатов разработки требованиям заказчика.

2. **Поддержка прототипирования.** Большинство моделей жизненного цикла, предназначенных для разработки сложного или критичного программного обеспечения, базируются на применении прототипирования. Разработка прототипов осуществляется на ранних этапах жизненного цикла и позволяет уточнять требования к программному обеспечению, а также прогнозировать поведение разрабатываемого продукта.

3. **Поддержка современных методологий разработки программного обеспечения.** Современные линейки CASE-средств поддерживают, как правило, различные методологии, предназначенные для использования на различных этапах процесса разработки. При этом выполняется графическая поддержка построения диаграмм различных типов, контроль корректности шагов проектирования и подготовка документации.

4. **Автоматическая генерация кода.** Кодогенерация позволяет построить автоматически до 90% исходных кодов на языках высокого уровня.

Различными CASE-средствами поддерживаются практически все известные языки программирования.

Использование CASE-средств позволяет улучшить качество создаваемых проектов, а при создании крупных корпоративных систем является практически неизбежным. Однако использование CASE-средств не освобождает проектировщика от понимания не только общей сущности, но и деталей логического проектирования.

Средства генерации кода

Для генерации кода должны быть определены метаданные, шаблоны и правила предметной области.
Правила предметной области определяют, какими будут шаблоны, метаданные. Генератор же на основе метаданных и шаблонов строит автоматически сгенерированный код.

Генерация кода *— автоматическое или автоматизированное создание программного кода специальными средствами (генераторами кода), при котором по заданным условиям полностью или частично формируется исходный код программы.*

Средства кодогенерации можно подразделить на два **вида**:
1) ***средства генерации управляющей структуры*** продукта; данные средства выполняют автоматическое построение логической структуры программного средства, кодов для базы данных, файлов, экранов, отчетов. Остальные фрагменты программного средства кодируются вручную;
2) ***средства генерации полного продукта***; данные средства позволяют на основе разработанных спецификаций или моделей генерировать полные коды программного средства, пользовательскую и программную документацию к нему.

Генераторы лучше всего применимы в проблемной области, требующей выполнения множества коротких, схожих итераций, например базы данных, где один и тот же шаблон проектирования применяется много раз к множеству таблиц базы данных.

Структура каждой таблицы может быть уникальной, но правила и условия, применяемые к полям таблицы при разработке кода, почти одинаковы.

Практически наиболее применимыми областями являются:

- приложения баз данных (объекты бизнес логики, поддержка транзакций);
- SQL-запросы;
- приложения, взаимодействующие с базами данных, использующие их для ввода-вывода данных;
- классы и структуры объектно-ориентированных языков программирования;
- тестирование программного кода;
- техническая документация на программу.

Применение автоматической генерации кода в разработке программного проекта дает ряд **новых возможностей**.

1. **Структурированность и согласованность кода**, сгенерированного машиной. Генератор позволяет с точностью до символа соблюдать правила написания кода.
2. **Согласованность на уровне архитектуры приложения.** Применение генератора дает уверенность в том, что создаваемый код соответствует изначально спроектированной структуре приложения и шаблонам. Если какую-то функциональность не получается реализовать с помощью генератора, это служит сигналом к тому, что она не соответствует изначальной архитектуре.
3. **Высокое качество кода и облегченное исправление ошибок.** Программа стабильна и свободна от ошибок.
4. **Гибкость к изменениям.** Если требования поменяются, можно только обновить шаблоны и сделать вывод новой версии кода. Есть возможность относительно легко и безболезненно переходить на другие платформы и технологии. Сводятся к минимуму ошибки, возникающие из-за человеческого фактора.
5. **Скорость разработки.** Если метаданные уже определены и введены, то генерация огромного объема кода может производиться за невероятно короткое время.
6. **Единый источник информации о приложении.** В приложениях, которые пишутся вручную, простое изменение имени таблицы или столбца может повлечь за собой серию изменений во многих участках кода. При генерации же метаданные находятся в отдельном источнике, поэтому достаточно изменить название таблицы или столбца в том месте, где определяется схема, и пересоздать код.
7. **Наглядность бизнес-логики.** В приложениях, которые пишутся вручную, смысл бизнес-логики теряется в сотнях и тысячах строк программного кода. Генераторы же используют файлы метаданных, в которых видна вся структура программируемой предметной области, ее поведение, исключения и т. д. Гораздо легче понять, что делает приложение на высоком уровне абстракции. Метаданные могут использовать эксперты в предметной области приложения.
8. **Усовершенствованный процесс разработки кода.** Теперь появляется возможность больше времени уделять проектированию приложения, так как процесс создания кода будет занимать меньше времени. Уменьшается срок выполнения проекта, повышается удобство сопровождения проекта. Также с помощью генератора можно быстро изготавливать множество прототипов кода, анализируя и тестируя которые можно избегать ситуаций, когда ошибка допущена на этапе проектирования, а выявлена была уже в процессе разработки.
9. **Возможность концентрации программистов на более интеллектуальных областях разработки**, что позволяет лучше использовать их профессиональные навыки, быстрее повышать квалификацию.
10. **Улучшение удобства работы программистов.** Код становится намного чище и проще, так как генерируется только то, что требуется сейчас.
11. **Улучшение мотивации и интереса к работе у программистов.** Все вышеперечисленное делает работу разработчика более интересной, насыщенной и продуктивной, что не может не сказываться положительно на мотивации, а это является очень важным фактором, влияющим на успех проекта.

Применение генератора кода может быть эффективно не во всех случаях.
Если объем проекта недостаточно большой, если код сложно поддается генерации, может оказаться, что легче написать приложение вручную. На практике всегда присутствует часть кода, которую нужно создать вручную. Объем такого кода может быть различным на разных проектах.
Генерация может осуществляться с нуля, тогда код генерируется из шаблонов и метаданных, предварительно созданного кода не существует. Иногда каким-либо образом изменяется уже существующий код. В этом случае первоначальный код играет роль шаблона, а также может содержать метаданные.
Метаданные могут храниться (и извлекаться) в специализированных файлах с иерархической структурой (например, XML-файлы), файлах языков проектирования (например, UML), других файловых источников, текстовых файлах, базах данных и др.
Сгенерированный код может быть завершенным и незавершенным. Незавершенный сгенерированный код нуждается в дальнейшей ручной модификации. Часто это является показателем того, что генератор недоработанный, сырой либо код плохо поддается генерации. Завершенный сгенерированный код не нуждается в модификации и его можно выложить в производственную схему. Это не означает, что программный проект не содержит вручную созданного кода, просто сгенерированный код интегрируется без изменений с вручную созданным кодом.

Масштаб генерации программного кода может быть различным:

- определенный модуль или группа файлов — может применяться при первоначальном пробном тестировании работы генератора;
- уровень в архитектуре приложения — генерируется только один уровень в архитектуре приложения. Очень часто это уровень базы данных, так он легче всего поддается генерации и от него зависит работа всех остальных уровней. Генерация других уровней без уровня базы данных потеряет много в своей эффективности;
- все уровни в архитектуре приложения — на каждом уровне приложения применяется сгенерированный код. Применяется, когда выработаны шаблоны, позволяющие сгенерировать большую часть приложения.

Генерация пользовательского интерфейса

Генерация пользовательского интерфейса предполагает следующее.

1. **Возможность абстрагировать бизнес-процессы от конкретной реализации пользовательского интерфейса.** Впоследствии это позволит быстрее переходить к новым технологиям и платформам, изменять или внедрять новые бизнес-процессы. Абстрактная форма независима от технологии и остается прежней. С минимальными изменениями ее можно использовать в генерации кода для новой технологии. Это является очень важным свойством, так как технологии разработки пользовательского интерфейса развиваются и меняются весьма быстро, по сравнению, например, с технологиями баз данных.
2. **Возможность систематизировать пользовательский интерфейс и сделать его единообразным во всем приложении.** Это относится как к внешнему виду и функциональности, так и к внутреннему содержимому, организации структуры кода и соответствия ее стандартам.
3. **Скорость внедрения.** Изменения могут быть быстро распространены по всему приложению, достаточно либо изменить шаблоны, либо изменить абстрактный файл определений.

4. **Гибкость к изменениям.** Разделение метаданных и шаблонов, а также большая скорость внедрения изменений при генерации позволяют легко и быстро модифицировать приложения. Можно внедрять несколько вариантов исполнения пользовательских операций, тем самым формируя богатый функциональный интерфейс.
5. **Существенное снижение числа ошибок.** Сведен к минимуму человеческий фактор при работе с кодом пользовательского интерфейса, содержащего еще множество вызовов и команд, относящихся к другим уровням приложения.

Для эффективной генерации пользовательского интерфейса следует в первую очередь минимизировать объем кода, не относящегося к пользовательскому интерфейсу. Его следует минимизировать в объеме путем оптимизации, а также создания и вызова соответствующих процедур в библиотеках функций. Также следует создавать максимально структурированные и хорошо иерархически организованные шаблоны. Все это позволяет иметь интерфейс, имеющий простую, стандартизированную и, соответственно, удобную для генерации структуру.

Следует заметить, что если изначально интерфейс плохо спроектирован и не продуман, слабо стандартизирован, то такие его качества будут автоматически воспроизводиться генератором. Хорошо продуманный интерфейс будет генерироваться так же, как если бы был сделан вручную.

Генерация документации

Разработка программного обеспечения подразумевает также создание и ведение технической документации на нее. Разработка полной и актуальной документации является весьма трудоемким процессом и занимает немало времени.

Также весомой проблемой при ведении технической документации на программный код приложения является рассогласованность кода приложения и самой документации.

Она возникает вследствие внесений модификаций в код приложения, в результате которого изменения не отображаются в документации полностью или частично.

Несмотря на то, что существует правило — *после каждого изменения в коде необходимо обновлять соответственно и документацию*, в реальности соблюдение подобного процесса работы требует немало усилий.

Кроме того, когда дело касается деталей, программисты больше предпочитают разбираться с программным кодом, нежели читать документацию, которая к тому же может быть неточной.

Из этого следует, что еще одной причиной неудовлетворительного применения документации является то, что она хранится в разных источниках и при обновлении приложения далеко не всегда соответственно изменяется документация.

Решением проблемы является *хранение кода и документации в одном источнике*.

Хорошо документированный код будет способствовать хорошему качеству программы, как в смысле написания кода, так и в смысле реализации логики проекта.

Когда документация и код находятся совместно в одном источнике, это улучшает понятность кода, возможности и удобство постоянного сопровождения документации программы в течение всей жизни программного обеспечения. Код, техническая документация и комментарии управляются как единое целое, не происходит их рассогласованности.

Генерация документации

Генератор документации применяется двумя **способами**:
1) документация *генерируется одновременно* с кодом исходя из определений метаданных, причем частью метаданных должны быть комментарии, которые будут включены как в сам код, так и в документацию;
2) из программного кода *выбираются важные блоки кода и комментарии*, из этой информации составляется документация. Этот вариант очень удобен, так как документация находится там же, где и код.

Одним из ключевых моментов для ведения актуальной и полной документации является грамотная ***организация системы комментариев***.

Возможности генерации документации:

- документация становится в большей степени согласованной с программным кодом, т. е. более точной и актуальной;
- экономится значительное количество времени и усилий, требуемых для разработки документации. Освобождаемые время и ресурсы можно применить для улучшения качества документации;
- более простым становится процесс изменения формата или стиля подачи документации. При наличии собственного генератора не надо менять формат каждого документа, достаточно изменить шаблон;
- документация формируется значительно быстрее.

Классификация CASE-средств

Все современные CASE-средства могут быть классифицированы в основном по типам и категориям. Классификация по типам отражает функциональную ориентацию CASE-средств. Классификация по категориям определяет степень интегрированности по выполняемым функциям.

Классификация по типам отражает функциональное назначение CASE-средств в жизненном цикле программного обеспечения.

1. **Средства анализа и проектирования.** Средства этого типа используются для поддержки начальных этапов процесса разработки: анализа предметной области, разработки требований к системе, проектирования системной архитектуры, разработки требований к программным средствам, проектирования программной архитектуры, технического проектирования программного обеспечения. Средства данного типа поддерживают известные методологии анализа и проектирования. На выходе генерируются спецификации системы, ее компонентов и интерфейсов, связывающих эти компоненты, архитектура системы, архитектура программного средства, технический проект программного средства, включая алгоритмы и определения структур данных.
2. **Средства проектирования баз данных и файлов.** Средства этого типа обеспечивают логическое моделирование данных, автоматическое преобразование моделей данных в третью нормальную форму, автоматическую генерацию схем баз данных и описаний форматов файлов на уровне программного кода.
3. **Средства для программирования и тестирования.** Данные средства выполняют автоматическую кодогенерацию программного обеспечения на основе спецификаций или моделей. Содержат графические редакторы, средства поддержки работы с репозиторием, генераторы и анализаторы кодов, генераторы тестов, анализаторы покрытия тестами, отладчики.
4. **Средства сопровождения.** Общей целью средств этого типа является поддержка корректировки, изменения, преобразования существующей системы, поддержка документации по проекту. К данным средствам относятся средства документирования, анализаторы программ, средства управления изменениями и конфигурацией ПО, средства реструктурирования, средства обеспечения мобильности, позволяющие перенести разработанную систему или программные средства в новое операционное или аппаратное окружение.
5. **Средства управления проектом.** К средствам данного типа относятся средства поддержки процесса управления жизненным циклом ПО. Их функциями являются планирование, контроль, руководство, организация взаимодействия и т. д.

Классификация по категориям отражает уровень интегрированности CASE-средств по выполняемым функциям.

1. **Категория Tool** (tool — рабочий инструмент) включает средства самого низкого уровня интегрированности. В данную категорию средств входят инструментальные средства, решающие небольшую автономную задачу при разработке программного средства или системы. Обычно средства данной категории являются компонентами CASE-средств более высокого уровня интегрированности.
2. **Категория ToolKit** (toolkit — набор инструментов, пакет разработчика) включает CASE-средства среднего уровня интегрированности. Средства данной категории используют репозиторий для всей информации о проекте и ориентированы обычно на поддержку одного этапа или одной работы процесса разработки или на поддержку одного из вспомогательных или организационных процессов жизненного цикла программных систем. CASE-средства данной категории представляют собой интегрированную совокупность инструментальных средств, имеющих, как правило, общую функциональную ориентацию.
3. **Категория Workbench** (workbench — рабочее место) обладает самой высокой степенью интеграции. Средства представляют собой интегрированную совокупность инструментальных средств, поддерживающих практически весь процесс разработки и ряд вспомогательных и организационных процессов жизненного цикла ПО. Используют репозиторий для хранения информации по проекту, поддерживают организацию коллективной работы над проектом. Обычно к категории Workbench относятся линейки CASE-средств при их интегральном использовании.

Классификация по уровням связана с областью действия CASE-средств в жизненном цикле программного обеспечения систем и организаций.

1. **Верхние** (Upper) CASE-средства — средства компьютерного планирования. Их основной целью является помощь руководителям организаций, предприятий и конкретных проектов в определении политики организации и создании планов проекта. CASE-средства данного уровня позволяют строить модель предметной области, проводить анализ различных сценариев (существующего, наилучших, наихудших), накапливать информацию для принятия оптимальных решений.
2. **Средние** (Middle) CASE-средства поддерживают начальные этапы процесса разработки (анализ предметной области, разработка требований к системе, проектирование системной архитектуры, разработка требований к программам, проектирование программной архитектуры). Обычно данные средства обладают возможностями накопления и хранения информации по проекту. Это позволяет использовать накопленные данные как в текущем, так и в других проектах. Зачастую поддерживают прототипирование и автоматическое документирование.
3. **Нижние** (Lower) CASE-средства поддерживают вторую половину работ процесса разработки ПО. Спецификации представляются обычно в виде моделей, которые непосредственно преобразуются в программные коды разрабатываемого программного средства или системы. Автоматически генерируется до 90% кодов. Входной информацией для кодогенераторов являются спецификации, разработанные как в CASE-средствах данного уровня, так и в CASE-средствах среднего уровня. CASE-средства нижнего уровня, как правило, поддерживают также прототипирование, тестирование, управление конфигурацией, генерацию документации, облегчают модификацию и сопровождение ПО.